

Министерство науки и высшего образования Российской

Федерации

Федеральное государственное автономное образовательное

учреждение

высшего образования «Национальный исследовательский

университет

«Московский институт электронной техники»

Лабораторная работа №8

по дисциплине «Управление качеством программного

обеспечения»

«Метрики Абреу»

Москва, 2024

Для успешной сдачи лабораторной работы необходимо выполнить одно задание по варианту.

На максимальный балл:

(«Номер компьютера» - 1) % 10 + 1 = **Номер задания**

Первый – Задание №1, Второй – Задание №2 и т.д.

Одиннадцатый – Задание №1, Двенадцатый – Задание №2 и т.д.

Задания делаются по образцу из примеров и оформляются в качестве отчётов. Перед сдачей необходимо внимательно проверить правильность выполнения, чтобы избежать понижения баллов.

Срок выполнения лабораторной работы с сохранением баллов – 2 недели.

Используемые языки: C, C++, C#, Java, Python

4.4. Метрики Абреу

4.4.1. Теоретические сведения

Свой подход к оценке качества объектно-ориентированного программирования разработал Фернандо Абреу (Fernando Brito e Abreu), предложив в 1994 г. набор метрик *MOOD* (Metrics for Object Oriented Design). Основными целями предложенного Абреу набора метрик являются следующие:

- учет базовых механизмов объектно-ориентированного подхода к созданию ПО (инкапсуляция, наследование, полиморфизм, посылка сообщений);

- формальное определение такого набора метрик, который позволит исключить или значительно снизить субъективность определения показателей качества ПС;

- независимость от размера оцениваемого программного продукта;

- независимость от языка программирования, который использовался при создании оцениваемого ПС.

Набор метрик Абреу включает в себя следующие показатели качества программного продукта:

- фактор закрытости метода (*MHF*);
- фактор закрытости свойства (*AHF*);
- фактор наследования метода (*MIF*);
- фактор наследования свойства (*AIF*);
- фактор полиморфизма (*POF*);
- фактор сцепления (*COF*).

Каждая метрика Абреу относится к одному из основных механизмов объектно-ориентированного подхода к программированию: инкапсуляции (метрики *MHF* и *AHF*), наследованию (метрики *MIF* и *AIF*), полиморфизму (*POF*) и отсылке сообщений (*COF*). В определениях набора *MOOD* не используются специфические конструкции языков программирования, что дает возможность исключить зависимость от языка программирования, применяемого при разработке программного средства.

Фактор закрытости метода *MHF* (Method Hiding Factor). Закрытость метода характеризует процентное количество классов, из которых конкретный метод невидим.

Для характеристики данной метрики вводятся следующие обозначения:

- $M_v(C_i)$ – интерфейс класса, определяющий количество видимых методов в классе C_i ;
- $M_h(C_i)$ – реализация класса, характеризующая количество скрытых методов в классе C_i ;
- $M_d(C_i) = M_v(C_i) + M_h(C_i)$ – общее количество методов, определенных в классе C_i , причем в общем количестве методов не учитываются унаследованные методы.

Тогда формула определения значения метрики MHF будет иметь следующий вид:

$$MHF = \frac{\sum_{i=1}^{TC} M_h(C_i)}{\sum_{i=1}^{TC} M_d(C_i)}, \quad (4.4.1)$$

где TC – количество классов в создаваемой программной системе.

Если уровень видимости m -го метода i -го класса из j -го класса определить как

$$is_visible(M_{mi}, C_j) = \begin{cases} 1, & \text{если } j \neq i \text{ и } C_j \text{ может вызвать } M_{mi}, \\ 0 & \text{в других случаях} \end{cases}, \quad (4.4.2)$$

а процентное количество классов, которые способны видеть m -й метод i -го класса, вычислить следующим образом:

$$V(M_{mi}) = \frac{\sum_{j=1}^{TC} is_visible(M_{mi}, C_j)}{TC - 1}, \quad (4.4.3)$$

то значение метрики закрытости метода MHF можно рассчитать с помощью следующего выражения:

$$MHF = \frac{\sum_{i=1}^{TC} \sum_{m=1}^{M_d(C_i)} (1 - V(M_{mi}))}{\sum_{i=1}^{TC} M_d(C_i)}. \quad (4.4.4)$$

Числитель формулы (4.4.4) содержит сумму закрытости всех методов во всех классах программного продукта, а знаменатель этого

выражения – общее количество методов, определенных в рассматриваемой системе. С ростом значения метрики MHF уменьшается плотность дефектов в системе, что означает и сокращение затрат, необходимых для их устранения. Обычно разработка класса программы представляет собой пошаговый процесс, при котором к каждому классу постепенно добавляется все больше и больше деталей (которые означают скрытые методы). Такая схема разработки способствует росту как значения метрики MHF , так и качества класса.

Фактор закрытости свойства AHF (Attribute Hiding Factor). Закрытость свойства представляет собой процентное количество классов, из которых данное свойство невидимо. По аналогии с предыдущей метрикой вводятся следующие обозначения:

- $A_v(C_i)$ – интерфейс класса, означающий количество видимых свойств в классе C_i ;

- $A_h(C_i)$ – реализация класса, которая характеризует количество скрытых свойств в классе C_i ;

- $A_d(C_i) = A_v(C_i) + A_h(C_i)$ – общее количество свойств, определенных в классе C_i , причем унаследованные свойства не учитываются.

При таких обозначениях выражение для определения метрики AHF примет следующий вид:

$$AHF = \frac{\sum_{i=1}^{TC} A_h(C_i)}{\sum_{i=1}^{TC} A_d(C_i)}, \quad (4.4.5)$$

где TC – количество классов в создаваемой программной системе.

Если видимость m -го свойства i -го класса из j -го класса вычислить в соответствии с выражением

$$is_visible(A_{mi}, C_j) = \begin{cases} 1, & \text{если } j \neq i \text{ и } C_j \text{ может вызвать } A_{mi}, \\ 0 & \text{в других случаях} \end{cases} \quad (4.4.6)$$

а процентное количество классов, которые видят m -е свойство i -го класса, определить следующим образом:

$$V(A_{mi}) = \frac{\sum_{i=1}^{TC} is_visible(A_{mi}, C_j)}{TC - 1}, \quad (4.4.7)$$

то выражение для определения значения метрики AHF можно записать так:

$$AHF = \frac{\sum_{i=1}^{\pi} \sum_{m=1}^{A_i(C_i)} (1 - V(A_{mi}))}{\sum_{i=1}^{\pi} A_d(C_i)}. \quad (4.4.8)$$

В числителе этой формулы AHF расположена сумма закрытости всех свойств во всех классах, а в знаменателе – общее количество свойств, определенных в анализируемой системе.

В идеальном случае все свойства должны быть скрыты и доступны только для методов соответствующего класса, т. е. наилучшим показателем является значение $AHF = 100\%$.

Фактор наследования метода MIF (Method Inheritance Factor). Для определения этой метрики Абреу вводит следующие обозначения:

- $M_i(C_i)$ – количество унаследованных и непереопределенных методов в классе C_i ;
- $M_0(C_i)$ – количество унаследованных и переопределенных методов в классе C_i ;
- $M_n(C_i)$ – количество новых (неунаследованных и переопределенных) методов в классе C_i ;
- $M_d(C_i) = M_n(C_i) + M_0(C_i)$ – количество методов, определенных в классе C_i ;
- $M_a(C_i) = M_d(C_i) + M_i(C_i)$ – общее количество методов, доступных в классе C_i .

Тогда формула для определения метрики MIF будет такой:

$$MIF = \frac{\sum_{i=1}^{\pi} M_i(C_i)}{\sum_{i=1}^{\pi} M_a(C_i)}. \quad (4.4.9)$$

Числителем MIF является сумма унаследованных (и непереопределенных) методов во всех классах рассматриваемой системы, в знаменателе – общее количество доступных методов (локально определенных и унаследованных) для всех классов. Нулевое значение метрики MIF говорит о том, что в анализируемой системе отсутствует эффективное наследование (например, все унаследованные методы переопределены). С ростом значения метрики MIF уменьшается плотность дефектов, следовательно, можно ожидать сокращения затрат на исправление ошибок. Высокие значения метрики MIF (на

уровне 70–80%) приводят к обратному эффекту, однако это не всегда однозначно, потому необходима дополнительная экспериментальная проверка. Можно сделать более мягкий вывод: умеренное использование наследования является весьма подходящим средством для снижения плотности дефектов и сокращения затрат на доработку программного средства.

Фактор наследования свойства AIF (Attribute Inheritance Factor). Для определения этой метрики вводятся следующие обозначения:

- $A_i(C_i)$ – количество унаследованных и непереопределенных свойств в классе C_i ;
- $A_0(C_i)$ – количество унаследованных и переопределенных свойств в классе C_i ;
- $A_n(C_i)$ – количество новых (неунаследованных и переопределенных) свойств в классе C_i ;
- $A_d(C_i) = A_n(C_i) + A_0(C_i)$ – количество свойств, определенных в классе C_i ;
- $A_a(C_i) = A_d(C_i) + A_i(C_i)$ – общее количество свойств, доступных в классе C_i .

Тогда формула для расчета метрики *AIF* примет следующий вид:

$$AIF = \frac{\sum_{i=1}^n A_i(C_i)}{\sum_{i=1}^n A_a(C_i)} \quad (4.4.10)$$

Числитель *AIF* образует сумма унаследованных (и непереопределенных) свойств во всех классах рассматриваемой системы, а знаменатель – общее количество доступных свойств (локально определенных и унаследованных) для всех классов.

Фактор полиморфизма POF (Polymorphism Factor) определяется с учетом следующих обозначений:

- $M_0(C_i)$ – количество унаследованных и переопределенных методов в классе C_i ;
- $M_n(C_i)$ – количество новых (неунаследованных и переопределенных) методов в классе C_i ;
- $DC(C_i)$ – количество потомков класса C_i ;
- $M_d(C_i) = M_n(C_i) + M_0(C_i)$ – количество методов, определенных в классе C_i .

Тогда формула метрики *POF* примет вид:

$$POF = \frac{\sum_{i=1}^{\pi} M_o(C_i)}{\sum_{i=1}^{\pi} [M_n(C_i) \cdot DC(C_i)]}, \quad (4.4.11)$$

Числитель метрики POF указывает на реальное количество возможных полиморфных ситуаций. Очевидно, что сообщение, посланное в класс C_i , статически или динамически связывается с реализацией именуемого метода. А этот метод, в свою очередь, может или быть представлен несколькими «формами», или переопределяться (в потомках C_i). Знаменатель метрики POF представляет максимальное количество возможных полиморфных ситуаций для класса C_i . Это происходит тогда, когда все новые методы, определенные в классе C_i , переопределяются во всех его потомках.

Умеренное использование полиморфизма уменьшает как плотность дефектов, так и затраты на доработку ПС. Однако при значениях метрики $POF > 10\%$ возможен обратный эффект, когда дефекты и затраты могут возрасти.

Фактор сцепления COF (Coupling Factor). В наборе метрик Абреу сцепление характеризует наличие между классами отношения «клиент–поставщик». Отношение «клиент–поставщик» ($C_c \rightarrow C_s$) здесь означает, что класс-клиент содержит по меньшей мере одну неунаследованную ссылку на свойство или метод класса-поставщика.

Если наличие отношения «клиент–поставщик» определять по формуле

$$is_client(C_c, C_s) = \begin{cases} 1, & \text{если } C_c \rightarrow C_s \cap C_c \neq C_s, \\ 0 & \text{в других случаях} \end{cases} \quad (4.4.12)$$

то формула для вычисления метрики COF примет следующий вид:

$$COF = \frac{\sum_{i=1}^{\pi} \left[\sum_{j=1}^{\pi} is_client(C_i, C_j) \right]}{TC^2 - TC}. \quad (4.4.13)$$

Знаменатель COF соответствует максимально возможному количеству сцеплений в системе с TC -классами, поскольку каждый класс может быть потенциальным поставщиком для других классов. Из приведенного рассмотрения исключены рефлексивные отношения, когда класс является собственным поставщиком. Числитель COF характеризует реальное количество сцеплений, не относящихся к наследованию.

С увеличением сцепления классов плотность дефектов и уровень затрат на доработку программы также возрастают. Сцепления отрицательно влияют на качество программного средства, надо добиваться, чтобы их было как можно меньше. Практическое применение этой метрики еще раз доказывает, что сцепление увеличивает сложность ПО, уменьшает инкапсуляцию и возможность повторного использования программных компонентов, затрудняет понимание и усложняет сопровождение ПС.

4.4.2. Задача «Платеж за электроэнергию»

Необходимо определить класс, описывающий платеж за электроэнергию. В рамках класса следует предусмотреть следующие поля:

- фамилия плательщика;
- потребление электроэнергии за оплачиваемый месяц;
- нормативное среднemesячное потребление;
- тариф (стоимость одного киловатт-часа).

Рекомендуется применять следующие методы:

- вычисление суммы оплаты;
- формирование сводной информации по одному платежу (вид платежа, фамилия, сумма, потребление).

Платеж может выполняться по показаниям счетчика или по нормативно установленному среднemesячному потреблению. В процессе эксплуатации программы тариф и нормативно установленное среднemesячное потребление не изменяются. Для создания конкретного платежа предусмотреть соответствующий конструктор.

Все платежи должны сохраняться в архиве. Запросы по ведению архива выполняются статическими методами класса «Запрос»:

- занесение платежей в архив. Данные платежа вводятся с клавиатуры. Ввод отрицательного показания счетчика означает оплату по нормативно установленному среднemesячному потреблению;

- вывод сводной информации из архива.

Архив моделируется массивом объектов.

В основном классе «Платежи» следует сформировать архив платежей. По данным архива предусмотреть выдачу сводной информации о платежах.

При решении задачи необходимо разработать исходный код программы, а также определить оценки характеристик программы на основе объектно-ориентированных метрик Абреу.

Текст программы на языке C# для реализации возможного алгоритма решения поставленной задачи представлен на рис. 4.1.

Номера строк	Строки программы
1	using System;
2	using System.Collections.Generic;
3	using System.Text;
4	namespace EX1
5	{
6	class Электро
7	{
8	private static double тариф = 1.84;
9	private static int потреблениеСреднее = 300;
10	private string фамилия;
11	private int потребление;
12	public Электро(string фамилия)
13	{
14	this.фамилия = фамилия;
15	потребление = потреблениеСреднее;
16	}
17	public Электро(string фамилия, int текущее, int предыдущее)

```
18 {
19     this.фамилия = фамилия;
20     потребление = текущее - предыдущее;
21 }
22 public double Сумма()
23 {
24     return потребление * тариф;
25 }
26 public string Инфо()
27 {
28     return string.Format("{0,-20} {1,-20} {2,10:f2} {3,10:d6}",
29         "Электроэнергия", фамилия, Сумма(), потребление);
30 }
31 }
32 class Запрос
33 {
34     public static void Заполнить(Электро[] платеж)
35     {
36         string фам;
37         int счПред=0, счТек=0;
38         Console.Clear();
39         for (int i = 0; i < платеж.Length; i++)
40         {
41             Console.Write("Платеж "+ i + ": Фамилия -> ");
42             фам = Console.ReadLine();
43             Console.Write("Платеж "+ i + ": Текущее значение счетчика -> ");
44             счТек = int.Parse(Console.ReadLine());
45             if (счТек > 0)
46             {
47                 Console.Write("Платеж "+ i + ": Предыдущее значение счетчика -> ");
48                 счПред = int.Parse(Console.ReadLine());
49             }
50             if(счТек <= 0)
51                 платеж[i] = new Электро(фам);
52             else
53                 платеж[i] = new Электро(фам, счТек, счПред);
54         }
55     }
56     public static void Вывести(Электро[] платеж)
57     {
58         for (int i = 0; i < платеж.Length; i++)
59             Console.WriteLine(платеж[i].Инфо());
60     }
61 }
62 class Платежи
63 {
```

```

64 static void Main(string[] args)
65 {
66     ConsoleKeyInfo rep;
67     Электро[] плз;
68     int кпз;
69     do
70     {
71         Console.Clear();
72         Console.Write("Количество платежей за электроэнергию: ");
73         кпз = int.Parse(Console.ReadLine());
74         плз = new Электро[кпз];
75         Запрос.Заполнить(плз);
76         Запрос.Вывести(плз);
77         Console.WriteLine("Для выхода нажмите ESC");
78         rep = Console.ReadKey(true);
79     } while(rep.Key != ConsoleKey.Escape);
80     }
81 }
82 }

```

Рис. 4.1. Пример реализации программы «Платеж за электроэнергию»

Реализация программы

Текст программы на языке C# для реализации возможного алгоритма решения поставленной задачи представлен на рис. 4.1.

Оценка характеристик программы

В соответствии с теорией Абреу необходимо определить значения следующих характеристик:

- фактора закрытости метода (*MHF*);
- фактора закрытости свойства (*AHF*);
- фактора наследования метода (*MIF*);
- фактора наследования свойства (*AIF*);
- фактора полиморфизма (*POF*);
- фактора сцепления (*COF*).

Каждая метрика Абреу относится к одному из основных механизмов объектно-ориентированного подхода к программированию: инкапсуляции (метрики *MHF* и *AHF*), наследованию (метрики *MIF* и *AIF*), полиморфизму (*POF*) и отсылке сообщений (*COF*). В определениях набора метрик Абреу не используются специфические конструкции языков программирования, что дает возможность исключить зависимость от языка программирования, применяемого при разработке программных средств.

Фактор закрытости метода *MHF* (Method Hiding Factor). Закрытость метода определяется количеством классов, из которых указанный метод невидим. Значение данной метрики Абреу предлагает определять по соотношению (4.4.1).

В исходном коде решаемой задачи (см. рис. 4.1) определено три класса:

- *class Электро*;
- *class Запрос*;
- *class Платежи*.

Класс *Электро* в своем составе имеет четыре метода:

- *public Электро(string фамилия)*;
- *public Электро(string фамилия, int текущее, int предыдущее)*;
- *public double Сумма()*;
- *public string Инфо()*.

Все методы открыты, так как определены модификатором *public*. Следовательно, $M_h(\text{Электро}) = 0$, $M_d(\text{Электро}) = 4$.

Класс *Запрос* в своем составе имеет два метода:

- *public static void Заполнить (Электро[] платеж);*
- *public static void Вывести (Электро[] платеж).*

Все методы открыты, так как определены модификатором *public*.

Следовательно, $M_h(\text{Запрос}) = 0$, $M_d(\text{Запрос}) = 2$.

Класс *Платежи* в своем составе имеет один метод *Main()*, который по умолчанию является открытым. Следовательно, $M_h(\text{Платежи}) = 0$, $M_d(\text{Платежи}) = 1$.

Таким образом, значение характеристики:

$$MHF = \frac{\sum_{i=1}^n M_h(C_i)}{\sum_{i=1}^n M_d(C_i)} = \frac{0+0+0}{4+2+1} = 0.$$

Решение задачи не имеет закрытых методов.

Фактор закрытости свойства АНФ (Attribute Hiding Factor). Закрытость свойства (поля) представляет собой количество классов, из которых данное свойство (поле) невидимо. Значение данной метрики Абреу предлагает определять по соотношению (4.4.4).

В классе *Электро* определено четыре поля (см. рис. 4.1):

- *private static double тариф = 1.84;*
- *private static int потреблениеСреднее = 300;*
- *private string фамилия;*
- *private int потребление.*

Все перечисленные поля являются закрытыми, так как определены модификатором *private*. Следовательно, $A_h(\text{Электро}) = 4$, $A_d(\text{Электро}) = 4$.

В классах *Запрос* и *Платежи* поля не определены, следовательно, для них $A_h = 0$ и $A_d = 0$. В этом случае

$$AHF = \frac{\sum_{i=1}^n A_h(C_i)}{\sum_{i=1}^n A_d(C_i)} = \frac{4+0+0}{4+0+0} = 1.$$

Закрытость полей всех классов программы составляет 100%.

Фактор наследования метода MIF (Method Inheritance Factor). Для определения этой метрики Абреу предложил выражение (4.4.9). В решении задачи принцип наследования не реализован. Следовательно, расчет метрики *MIF* для данной задачи не имеет смысла.

Фактор наследования свойства AIF (Attribute Inheritance Factor) предложено определять в соответствии с формулой (4.4.10). В решении задачи принцип наследования не реализован. Следовательно, расчет метрики AIF не имеет смысла.

Фактор полиморфизма POF (Polymorphism Factor) рассчитывается в соответствии с выражением (4.4.11). В реализации данной задачи принцип наследования не реализован. Следовательно, расчет метрики POF не имеет смысла.

Фактор сцепления COF (Coupling Factor) вычисляется с использованием выражения (4.4.13).

Для класса *Электро* возможными классами-поставщиками могут быть классы *Запрос* и *Платежи* (см. рис. 4.1). В классе *Электро* не содержится ни одной ссылки на свойства и методы этих классов, поэтому $is_client(Электро, Запрос) = 0$; $is_client(Электро, Платежи) = 0$.

Для класса *Запрос* возможными поставщиками могут быть *Электро* и *Платежи*. Класс *Запрос* содержит ссылки на два поля и три метода класса *Электро* (см. рис. 4.1, строки 51, 53, 59). Ссылок на класс *Платежи* нет. Следовательно, $is_client(Запрос, Электро) = 1$; $is_client(Запрос, Платежи) = 0$.

Для класса *Платежи*: $is_client(Платежи, Электро) = 1$; $is_client(Платежи, Запрос) = 1$, так как в этом классе содержатся ссылки на поля и методы класса *Электро* (см. рис. 4.1, строка 74) и ссылки на методы класса *Запрос* (см. рис. 4.1, строки 75 и 76).

Таким образом, получаем:

$$COF = \frac{\sum_{i=1}^{TC} \left[\sum_{j=1}^{TC} is_client(C_i, C_j) \right]}{TC^2 - TC} = \frac{0+1+2}{3^2 - 3} = \frac{3}{6} = 0,5.$$

Фактор сцепления между классами невысок. Следовательно, возможная плотность дефектов ожидается низкой, уровень затрат на доработку программы будет небольшой.

4.4.3. Задача «Геометрия окружности и прямоугольника»

В предметной области «Геометрия» определены информационные объекты, описывающие понятия «Точка», «Окружность» и «Прямоугольник» в виде классов. Классы «Окружность» и «Прямоуголь-

ник» являются наследниками класса «Точка». Каждый из классов имеет метод вывода параметров объекта. Следует иметь в виду, что метод вывода в базовом классе не виртуальный и не абстрактный.

Методы вывода в наследниках переопределены. Вызываемый метод будет определяться типом ссылки.

Базовый класс «Точка» описан следующим образом:

- закрытые поля:
 - координата по оси X (вещественная);
 - координата по оси Y (вещественная);
- метод: вывод координат точки.

Класс «Окружность» задан следующими элементами:

- закрытые поля:
 - координаты центра – унаследованные поля;
 - радиус (вещественный);
- метод: вывод координат центра и длины окружности.

Класс «Прямоугольник» задается так:

- закрытые поля:
 - координаты левого верхнего угла – унаследованные поля;
 - основание (вещественное);
 - высота (вещественная);
- метод: вывод координат левого верхнего угла и площади.

В основном классе необходимо сформировать список геометрических объектов разных типов в виде массива и вывести параметры объектов на экран. Разработать исходный код программы, по которому определить оценки характеристик программы на основе объектно-ориентированных метрик Абреу.

Реализация программы

Текст программы на языке C# для реализации возможного алгоритма решения поставленной задачи представлен на рис. 4.2.

Оценка характеристик программы

Фактор закрытости метода MHF (Method Hiding Factor). Закрытость метода определяется количеством классов, из которых указанный метод невидим. Значение данной метрики Абреу предлагает определять по соотношению (4.4.1).

Номера строк	Строки программы
1	namespace EX1_1
2	{
3	class Точка
4	{
5	protected double x, y;
6	public Точка(double x, double y)
7	{
8	this.x = x; this.y = y;
9	}
10	public void Показать()
11	{
12	Console.WriteLine("Точка ({0:f2}:{1:f2})", x, y);
13	}
14	}
15	class Окружность: Точка
16	{
17	private double r;
18	public Окружность(double x, double y, double r): base(x, y)
19	{
20	this.r = r;
21	}
22	public new void Показать()
23	{
24	Console.WriteLine("Центр({0:f2}:{1:f2}) Длина={2:f2}", x, y, 2*Math.PI*r);
25	}
26	}
27	
28	class Прямоугольник: Точка
29	{
30	private double a, h;

```

31 public Прямоугольник(double x, double y, double a, double h): base(x, y)
32 {
33     this.a = a; this.h = h;
34 }
35 public new void Показать()
36 {
37     Console.WriteLine("Угол({0:f2}:{1:f2}) Площадь={2:f2}", x, y, a*h);
38 }
39 }
40 class Program
41 {
42     static void Main(string[] args)
43     {
44         Точка[] список = new Точка[6];
45         список[0] = new Точка(-1.0, -1.0);
46         список[1] = new Окружность(1.0, 1.0, 1.0);
47         список[2] = new Прямоугольник(10.0, 10.0, 10.0, 10.0);
48         список[3] = new Точка(-2.0, -2.0);
49         список[4] = new Окружность(3.0, 3.0, 3.0);
50         список[5] = new Прямоугольник(20.0, 20.0, 0.5, 0.5);
51         for (int i = 0; i < список.Length; i++) список[i].Показать();
52         Console.WriteLine();
53         for (int i = 0; i < список.Length; i++)
54         {
55             if (список[i] is Точка)
56                 ((Точка)список[i]).Показать();
57             else
58                 if (список[i] is Окружность)
59                     ((Окружность)список[i]).Показать();
60                 else
61                     if (список[i] is Прямоугольник)
62                         ((Прямоугольник)список[i]).Показать();
63         }
64         Console.WriteLine();
65         for (int i = 0; i < список.Length; i++)
66             switch (список[i].GetType().Name)
67             {
68                 case "Точка": ((Точка)список[i]).Показать(); break;
69                 case "Окружность": ((Окружность)список[i]).Показать(); break;
70                 case "Прямоугольник": ((Прямоугольник)список[i]).Показать(); break;
71             }
72     }
73 }
74

```

Рис. 4.2. Пример реализации программы «Геометрия»

В исходном коде решаемой задачи (см. рис. 4.2) определено четыре класса:

- *class Точка* – базовый класс, определяющий понятие точка, расположенная на координатной плоскости;
- *class Окружность* – производный класс от *class Точка*, определяющий понятие *окружность* с центром в точке, расположенной на координатной плоскости;
- *class Прямоугольник* – производный класс от *class Точка*, определяющий понятие *прямоугольник*, расположение которого на плоскости определяется координатами точки верхнего левого его угла;
- *class Program* – класс, в котором содержится основной метод обработки информации массивов объектов типа *Точка*, *Окружность* и *Прямоугольник*.

Класс *Точка* в своем составе имеет два открытых метода – *public Точка(double x, double y)* и *public void Показать()*, так как они определены модификатором *public*. Следовательно, $M_h(\text{Точка}) = 0$, $M_d(\text{Точка}) = 2$.

Класса *Окружность* включает два метода, из которых оба являются открытыми (см. рис. 4.2, строки 18, 22). Следовательно, $M_h(\text{Окружность}) = 0$, $M_d(\text{Окружность}) = 2$.

Класс *Прямоугольник* включает два метода, которые также являются открытыми (см. рис. 4.2, строки 31, 35). Следовательно, $M_h(\text{Прямоугольник}) = 0$, $M_d(\text{Прямоугольник}) = 2$.

Класс *Program* в своем составе имеет один метод *Main()*, который по умолчанию является открытым. Следовательно, $M_h(\text{Program}) = 0$, $M_d(\text{Program}) = 1$.

Таким образом, значение характеристики будет определяться так:

$$MHF = \frac{\sum_{i=1}^{\pi} M_h(C_i)}{\sum_{i=1}^{\pi} M_d(C_i)} = \frac{0+0+0+0}{2+2+2+1} = 0.$$

Решение задачи не имеет закрытых методов.

Фактор закрытости свойства AHF (Attribute Hiding Factor). Закрытость свойства (поля) представляет собой количество классов, из которых данное свойство (поле) невидимо. Значение данной метрики Абреу предлагает определять по соотношению (4.4.4).

В классе *Точка* определено два поля (см. рис. 4.2, строка 5). Эти поля являются закрытыми, так как определены модификатором *protected*. Такие поля имеют следующие значения:

- $A_h(\text{Точка}) = 2$;
- $A_d(\text{Точка}) = 2$.

В классе *Окружность* определено одно-единственное закрытое поле (см. рис. 4.2, строка 17), для которого можно записать:

- $A_h(\text{Окружность}) = 1$;
- $A_d(\text{Окружность}) = 1$.

В классе *Прямоугольник* определено два поля, оба являются закрытыми (см. рис. 4.2, строка 30):

- $A_h(\text{Прямоугольник}) = 2$;
- $A_d(\text{Прямоугольник}) = 2$.

В классе *Program* поля не определены. Таким образом, для иско-
мой метрики можно записать:

$$AHF = \frac{\sum_{i=1}^{TC} A_h(C_i)}{\sum_{i=1}^{TC} A_d(C_i)} = \frac{2+1+2+0}{2+1+2+0} = 1.$$

Закрытость полей всех классов программы составляет 100%.

Фактор наследования метода MIF (Method Inheritance Factor). Для определения этой метрики Абреу предложил выражение (4.4.9). В программе содержится один базовый класс *Точка* и два класса-наследника: *Окружность* и *Прямоугольник*.

Для класса *Точка* количество унаследованных методов равно 0, количество неунаследованных методов равно 2:

- $M_i(\text{Точка}) = 0$;
- $M_0(\text{Точка}) = 0$;
- $M_n(\text{Точка}) = 2$;
- $M_d(\text{Точка}) = M_n(\text{Точка}) + M_0(\text{Точка}) = 2+0 = 2$;
- $M_a(\text{Точка}) = M_d(\text{Точка}) + M_i(\text{Точка}) = 2+0 = 2$.

Для класса *Окружность* можно записать следующее:

- $M_i(\text{Окружность}) = 0$;
- $M_0(\text{Окружность}) = 1$;
- $M_n(\text{Окружность}) = 1$;
- $M_d(\text{Окружность}) = M_n(\text{Окружность}) + M_0(\text{Окружность}) = 1+1 = 2$;
- $M_a(\text{Окружность}) = M_d(\text{Окружность}) + M_i(\text{Окружность}) = 2+0 = 2$,

так как класс содержит один унаследованный переопределенный метод `public new void Показать()` и один неунаследованный метод `public Окружность(double x, double y, double r): base(x, y)`.

Для класса *Прямоугольник* характеристики будут равны:

- $M_i(\text{Прямоугольник}) = 0$;
- $M_0(\text{Прямоугольник}) = 1$;
- $M_n(\text{Прямоугольник}) = 1$;
- $M_d(\text{Прямоугольник}) = M_n(\text{Прямоугольник}) + M_0(\text{Прямоугольник}) = 1 + 1 = 2$;
- $M_a(\text{Прямоугольник}) = M_d(\text{Прямоугольник}) + M_i(\text{Прямоугольник}) = 2 + 0 = 2$,

так как класс содержит один унаследованный переопределенный метод `public new void Показать()` и один неунаследованный метод `public Прямоугольник(double x, double y, double a, double h): base(x, y)`.

Для класса *Program*:

- $M_i(\text{Program}) = 0$;
- $M_0(\text{Program}) = 0$;
- $M_n(\text{Program}) = 1$;
- $M_d(\text{Program}) = M_n(\text{Program}) + M_0(\text{Program}) = 1 + 0 = 1$;
- $M_a(\text{Program}) = M_d(\text{Program}) + M_i(\text{Program}) = 1 + 0 = 1$.

Таким образом, значение метрики *MIF* составляет:

$$MIF = \frac{\sum_{i=1}^{TC} M_i(C_i)}{\sum_{i=1}^{TC} M_a(C_i)} = \frac{0+0+0+0}{2+2+2+1} = 0.$$

Фактор наследования свойства AIF (Attribute Inheritance Factor) предложено определять в соответствии с формулой (4.4.10).

Для класса *Точка* количество унаследованных полей равно нулю:

- $A_i(\text{Точка}) = 0$;
- $A_0(\text{Точка}) = 0$;
- $A_n(\text{Точка}) = 2$;
- $A_d(\text{Точка}) = A_n(\text{Точка}) + A_0(\text{Точка}) = 2 + 0 = 2$;
- $A_a(\text{Точка}) = A_d(\text{Точка}) + A_i(\text{Точка}) = 2 + 0 = 2$.

Класс *Окружность* наследует от класса *Точка* два поля. В классе *Окружность* унаследованные поля не переопределяются, но дополнительно определяется одно новое поле (см. рис. 4.2, строка 17). Для класса *Окружность*:

- $A_i(\text{Точка}) = 2$;
- $A_0(\text{Точка}) = 0$;
- $A_n(\text{Точка}) = 1$;
- $A_d(\text{Точка}) = A_n(\text{Точка}) + A_0(\text{Точка}) = 1 + 0 = 1$;
- $A_a(\text{Точка}) = A_d(\text{Точка}) + A_i(\text{Точка}) = 1 + 2 = 3$.

Класс *Прямоугольник* наследует от класса *Точка* два поля. В классе *Прямоугольник* унаследованные поля не переопределяются, но дополнительно определяются два новых поля (см. рис. 4.2, строка 30).

Для класса *Прямоугольник*:

- $A_i(\text{Точка}) = 2$;
- $A_0(\text{Точка}) = 0$;
- $A_n(\text{Точка}) = 2$;
- $A_d(\text{Точка}) = A_n(\text{Точка}) + A_0(\text{Точка}) = 2 + 0 = 2$;
- $A_a(\text{Точка}) = A_d(\text{Точка}) + A_i(\text{Точка}) = 2 + 2 = 4$.

Класс *Program* не является наследником, и поля в нем не определены, тогда:

- $A_i(\text{Точка}) = 0$;
- $A_0(\text{Точка}) = 0$;
- $A_n(\text{Точка}) = 0$;
- $A_d(\text{Точка}) = A_n(\text{Точка}) + A_0(\text{Точка}) = 0 + 0 = 0$;
- $A_a(\text{Точка}) = A_d(\text{Точка}) + A_i(\text{Точка}) = 0 + 0 = 0$.

Таким образом, значение метрики *AIF* составляет:

$$AIF = \frac{\sum_{i=1}^{TC} A_i(C_i)}{\sum_{i=1}^{TC} A_a(C_i)} = \frac{0 + 2 + 2 + 0}{2 + 3 + 4 + 0} = \frac{4}{9} = 0,44.$$

Фактор полиморфизма *POF* (Polymorphism Factor) рассчитывается в соответствии с выражением (4.4.11).

Класс *Точка* является базовым и не имеет предков, он имеет двух потомков и не имеет унаследованных полей и методов. Для класса *Точка*:

- $M_0(\text{Точка}) = 0$;
- $M_n(\text{Точка}) = 2$;
- $DC(\text{Точка}) = 2$;
- $M_d(\text{Точка}) = M_n(\text{Точка}) + M_0(\text{Точка}) = 2 + 0 = 2$.

Класс *Окружность* является потомком класса *Точка* и не является базовым для других классов. В классе *Окружность* осуществляет-

ся переопределение наследуемого метода *public new void Показать()* от класса *Точка*. Для класса *Окружность* имеем следующие данные:

- $M_0(\text{Окружность})=1$;
- $M_n(\text{Окружность})=1$;
- $DC(\text{Окружность})=0$;
- $M_d(\text{Окружность})=M_n(\text{Окружность})+M_0(\text{Окружность})=1+1=2$.

Класс *Прямоугольник* является потомком класса *Точка* и не является базовым для других классов. В классе *Прямоугольник* осуществляется переопределение наследуемого метода *public new void Показать()* от класса *Точка*. Для класса *Прямоугольник* имеем следующие данные:

- $M_0(\text{Прямоугольник})=1$;
- $M_n(\text{Прямоугольник})=1$;
- $DC(\text{Прямоугольник})=0$;
- $M_d(\text{Прямоугольник})=M_n(\text{Прямоугольник})+M_0(\text{Прямоугольник})=1+1=2$.

Класс *Program* не является ни базовым, ни производным классом. Для класса *Program* имеем следующие характеристики:

- $M_0(\text{Program})=0$;
- $M_n(\text{Program})=1$;
- $DC(\text{Program})=0$;
- $M_d(\text{Program})=M_n(\text{Program})+M_0(\text{Program})=1+0=1$.

Значение метрики *POF* для программы в целом определится так:

$$POF = \frac{\sum_{i=1}^{TC} M_0(C_i)}{\sum_{i=1}^{TC} [M_n(C_i) \cdot DC(C_i)]} = \frac{0+1+1+0}{2 \cdot 2 + 1 \cdot 0 + 1 \cdot 0 + 1 \cdot 0} = \frac{2}{4} = 0,5.$$

Уровень полиморфизма программы составляет 50%, что может привести к повышенным трудозатратам при модификации программы.

Фактор сцепления *COF* (Coupling Factor) вычисляется с использованием выражения (4.4.13).

Для класса *Точка* возможными поставщиками могут быть классы *Окружность*, *Прямоугольник*, *Program*. В классе *Точка* отсутствуют свойства и методы других классов. Следовательно,

- $is_client(\text{Точка}, \text{Окружность})=0$;
- $is_client(\text{Точка}, \text{Прямоугольник})=0$;
- $is_client(\text{Точка}, \text{Program})=0$.

Для класса *Окружность* поставщиками могут быть классы *Точка*, *Прямоугольник*, *Program*. В классе *Окружность* ссылки на свойства и методы поставщиков отсутствуют. Следовательно,

- $is_client(Окружность, Точка) = 0$;
- $is_client(Окружность, Прямоугольник) = 0$;
- $is_client(Окружность, Program) = 0$.

Для класса *Прямоугольник* поставщиками могут быть классы *Точка*, *Окружность*, *Program*. В классе *Прямоугольник* ссылки на свойства и методы поставщиков отсутствуют. Следовательно,

- $is_client(Прямоугольник, Точка) = 0$;
- $is_client(Прямоугольник, Окружность) = 0$;
- $is_client(Прямоугольник, Program) = 0$.

Для класса *Program* поставщиками могут быть классы *Точка*, *Окружность*, *Прямоугольник*. В классе *Program* имеются ссылки на свойства и методы всех поставщиков. Следовательно,

- $is_client(Program, Точка) = 1$;
- $is_client(Program, Окружность) = 1$;
- $is_client(Program, Прямоугольник) = 1$.

Определяем значение метрики COF для решения в целом:

$$COF = \frac{\sum_{i=1}^{TC} \left[\sum_{j=1}^{TC} is_client(C_i, C_j) \right]}{TC^2 - TC} = \frac{0+0+0+3}{16-4} = \frac{3}{12} = 0,25.$$

Численное значение фактора сцепления между классами достаточно низкий. Следовательно, возможная плотность дефектов ожидается низкой, уровень затрат на доработку и модификацию программы будет небольшой.

4.4.4. Задачи для самостоятельного решения

В предлагаемых задачах необходимо разработать программу, реализующую решение по приведенному условию, а затем оценить характеристики разработанной программы на основе применения объектно-ориентированных метрик Абреу.

Задача 1. Разработать приложение для вычисления результирующей информации об объектах, описанных с помощью наследования:

- базовый объект – «Сетевое напряжение» (поля: напряжение – U , количество фаз – $faza$);

- производный объект 1 – «Электроустановка на 220 В» с полями: тип электроустановки – $type$, мощность – wt ;

- производный объект 2 – «трехфазная электроустановка на 360 В» с полями: тип электроустановки – $type$, сопротивление – R , потребляемый ток – amp .

Для решения задачи необходимо:

а). Определить базовый класс и производные классы, используя наследование.

б). Используя виртуальный метод печати базового класса, разработать переопределенные методы производных классов для вывода результирующей информации из перечня:

- тип электроустановки,

- сопротивление $R = U / I$;

- потребляемый ток $I = U / R$,

- мощность $W = I \cdot U$.

в). Создать массив для хранения ссылок на объекты, следующие в произвольном порядке. Всю необходимую информацию вводит пользователь.

г). Создать объекты, присвоив начальные значения полям объекта с помощью конструктора.

д). Используя массив ссылок и цикл, вывести результирующую информацию (см. требование б).

Задача 2. Разработать приложение для вычисления результирующей информации об объектах, описанных с помощью наследования:

- базовый объект – «Тарифный план сотовой связи» (поле: стоимость минуты разговора – sm);

- производный объект 1 – «Телефон МТС» с полями: тип сотовой связи – $Type$, сумма на счете – $schet$, особые условия);

- производный объект 2 – «Телефон Мегафон» с полями: тип сотовой связи – $Type$, сумма на счете – $schet$, количество бесплатных минут – $kolmin$.

Для решения задачи необходимо:

а). Определить базовый класс и производные классы, используя наследование.

б). Используя виртуальный метод печати базового класса, разработать переопределенные методы производных классов для вывода результирующей информации:

- тип сотовой связи;
- общая продолжительность разговора в рамках денежной суммы, хранящейся на счете.

в). Создать массив для хранения ссылок на объекты, следующие в произвольном порядке. Всю необходимую информацию вводит пользователь.

г). Создать объекты, присвоив начальные значения полям объекта с помощью конструктора.

д). Используя массив ссылок и цикл, вывести результирующую информацию (см. требование б).

Задача 3. Разработать приложение для вычисления результирующей информации об объектах, описанных с помощью наследования:

- базовый объект – «Стоимость 1 кВт·ч электроэнергии без льгот» (поля: стоимость 1 кВт·ч в дневное время – *sd*, стоимость 1 кВт·ч в ночное время – *sn*);

- производный объект 1 – «Квартира с электроплитой» с полями: дневной расход электроэнергии за месяц (количество киловатт) – *sdm*, ночной расход электроэнергии за месяц – *snm*, коэффициент снижения стоимости (льгота) – *koef*. Стоимость электроэнергии для квартир с электроплитами уменьшается в *koef* раз;

- производный объект 2 – «Квартира с газовой плитой» с полями: дневной расход электроэнергии за месяц – *sdmGaz*, ночной расход электроэнергии за месяц – *snmGaz*).

Для решения задачи необходимо:

а). Определить базовый класс и производные классы, используя наследование.

б). Используя виртуальный метод печати базового класса, разработать переопределенные методы производных классов для вывода результирующей информации: тип квартиры и плата за электроэнергию за месяц.

в). Создать массив для хранения ссылок на объекты, следующие в произвольном порядке. Всю необходимую информацию вводит пользователь.

г). Создать объекты, присвоив начальные значения полям объекта с помощью конструктора.

д). Используя массив ссылок и цикл, вывести на экран информацию о квартирах (см. требование б).

Задача 4. Разработать приложение для вычисления результирующей информации об объектах, описанных с помощью наследования:

- базовый объект – «Нефтехранилище» (поле: объем в тоннах – *vbase*);
- производный объект 1 – «Перевозка танкером» с полями: вместимость в тоннах – *vTanker*;
- производный объект 2 – «Перевозка железнодорожным составом» с полями: емкость цистерны в тоннах – *vCist*, количество цистерн в составе – *nCist*.

Для решения задачи необходимо:

- а). Определить базовый класс и производные классы, используя наследование.
- б). Используя виртуальный метод печати базового класса, разработать переопределенные методы производных классов для вывода результирующей информации: тип перевозки и количество танкеров, необходимых для вывоза всей нефти из хранилища, или количество железнодорожных составов для вывоза всей нефти из хранилища.
- в). Создать массив для хранения ссылок на объекты, следующие в произвольном порядке. Всю необходимую информацию вводит пользователь.
- г). Создать объекты, присвоив начальные значения полям объекта с помощью конструктора.
- д). Используя массив ссылок, вывести в цикле информацию (см. требование б).

Задача 5. Разработать приложение для вычисления результирующей информации об объектах, описанных с помощью наследования:

- базовый объект – «Бензобак» (поле: емкость бензобака – *Vb*);
- производный объект 1 – «Спортивный автомобиль» с полями: тип автотранспортного средства – *type*, средний расход бензина на 1 км пробега – *rb*, стоимость одного литра бензина – *slb*;
- производный объект 2 – «Автобус» с полями: тип автотранспортного средства – *type*, количество бензобаков – *kb*, средний расход бензина на 1 км пробега – *rb*, стоимость одного литра бензина – *slb*.

Для решения задачи необходимо:

- а). Определить базовый класс и производные классы, используя наследование.
- б). Используя виртуальный метод печати базового класса, разработать переопределенные методы производных классов для вывода

на экран результирующей информации: тип автотранспортного средства, средний пробег автотранспортного средства при полных бензобаках без дозаправки, стоимость заправки бензобаков.

в). Создать массив для хранения ссылок на объекты, следующие в произвольном порядке. Всю необходимую информацию вводить с клавиатуры.

г). Создать объекты, присвоив начальные значения полям объекта с помощью конструктора.

д). Используя массив ссылок и цикл, вывести на экран информацию (см. требование б).

Задача 6. Разработать приложение для вычисления и вывода на экран результирующей информации об объектах, описанных с помощью наследования:

- базовый объект – «Строительный блок кубической формы» (поле: длина стороны блока – *lblock*);

- производный объект 1 – «Забор толщиной в 1 блок» с полями: длина забора – *lZab*, высота забора – *hZab*;

- производный объект 2 – «Ленточный фундамент прямоугольной формы толщиной в 1 блок» с полями: длина фундамента – *lFund*, ширина фундамента – *bFund*, высота фундамента – *hFund*.

Для решения задачи необходимо:

а). Определить базовый класс и производные классы, используя наследование.

б). Используя виртуальный метод печати базового класса, разработать переопределенные методы производных классов для вывода на экран результирующей информации: тип объекта и количество строительных блоков, необходимых для строительства объекта. Количество блоков округлять в большую сторону.

в). Создать массив для хранения ссылок на объекты, следующие в произвольном порядке. Всю необходимую информацию вводить с клавиатуры.

г). Создать объекты, присвоив начальные значения полям объекта с помощью конструктора.

д). Используя массив ссылок и цикл, вывести на экран информацию (см. требование б).

Задача 7. Разработать приложение для вычисления и вывода на экран результирующей информации об объектах, описанных с помощью наследования:

- базовый объект – «Топливный бак» (поле: емкость бака – Vb);
- производный объект 1 – «Самолет» с полями: тип самолета – $typea$, средний расход керосина на 1 км пути – rka , предполагаемое расстояние перелета – km , тип топлива – $typef$);
- производный объект 2 – «Вертолет» с полями: тип вертолета – $typev$, средний расход бензина на 1 км пути – rkv , предполагаемое расстояние перелета – km , тип топлива – $typef$.

Для решения задачи необходимо:

а). Определить базовый класс и производные классы, используя наследование.

б). Используя виртуальный метод печати базового класса, разработать переопределенные методы производных классов для вывода результирующей информации: тип летательного аппарата, сведения о возможности (невозможности) перелета на одной заправке на заданное расстояние и количестве дозаправок.

в). Создать массив для хранения ссылок на объекты, следующие в произвольном порядке. Всю необходимую информацию вводит пользователь.

г). Создать объекты, присвоив начальные значения полям объекта с помощью конструктора.

д). Используя массив ссылок и цикл, вывести информацию (см. требование б).

Задача 8. Разработать приложение для вычисления результирующей информации об объектах, описанных с помощью наследования:

- базовый объект – «Проходной балл» (поле: проходной балл – $ball$, название факультета – fak);

- производный объект 1 – «Абитуриент факультета 1» с полями: фамилия абитуриента – Fam , оценка по русскому языку – BR , оценка по иностранному языку – BF , оценка по литературе – BL);

- производный объект 2 – «Абитуриент факультета 2» с полями: фамилия абитуриента – Fam , оценка по физике – $BFiz$, оценка по математике – BM , оценка по информатике – BI).

Для решения задачи необходимо:

а). Определить базовый класс и производные классы, используя наследование.

б). Используя виртуальный метод печати базового класса, разработать переопределенные методы производных классов для вывода

результатирующей информации: фамилия абитуриента, информация о поступлении или отчислении.

в). Создать массив для хранения ссылок на объекты, следующие в произвольном порядке. Всю необходимую информацию вводит пользователь.

г). Создать объекты, присвоив начальные значения полям объекта с помощью конструктора.

д). Используя массив ссылок и цикл, вывести информацию (см. требование б).

Задача 9. Разработать приложение для вычисления результирующей информации об объектах, описанных с помощью наследования:

- базовый объект – «Тариф страхования на год» (поле: тариф – *tarif*);
- производный объект 1 – «Сумма страховки начинающего водителя» с полями: фамилия – *name*, срок страхования – *time*, повышающий коэффициент – *koefup*);
- производный объект 2 – «Сумма страховки опытного водителя» с полями: фамилия – *name*, срок страхования – *time*, понижающий коэффициент – *koefd*).

Для решения задачи необходимо:

а). Определить базовый класс и производные классы, используя наследование.

б). Используя виртуальный метод печати базового класса, разработать переопределенные методы производных классов для вывода результирующей информации: фамилия водителя, сумма страховой премии на заданный период.

в). Создать массив для хранения ссылок на объекты, следующие в произвольном порядке. Всю необходимую информацию вводит пользователь.

г). Создать объекты, присвоив начальные значения полям объекта с помощью конструктора.

д). Используя массив ссылок и цикл, вывести информацию (см. п. б).

Задача 10. Разработать приложение для вычисления результирующей информации об объектах, описанных с помощью наследования:

- базовый объект – «Товар» (поле: стоимость единицы товара – *цена*, единица стоимости (шт. или кг) – *Item*);
- производный объект 1 – «Продукт питания» с полями: вес – *vProdukt*, название продукта – *Name*);

• производный объект 2 – «Одежда» с полями: количество – *nOdejda*, размер – *lOdejda*.

Для решения задачи необходимо:

а). Определить базовый класс и производные классы, используя наследование.

б). Используя виртуальный метод печати базового класса, разработать переопределенные методы производных классов для вывода результирующей информации: тип товара, информация о товаре и стоимость покупки.

в). Создать массив для хранения ссылок на объекты, следующие в произвольном порядке. Всю необходимую информацию вводит пользователь.

г). Создать объекты, присвоив начальные значения полям объекта с помощью конструктора.

д). Используя массив ссылок и цикл, вывести информацию (см. требование б).